Batch: B2	Roll No.: 16010124107
Experiment No. 04	
Grade: AA / AB / BB / BC / CC / CD /DD	
Signature of the Staff In-charge with date	

TITLE: CPU Scheduling – Non Preemptive

AIM: Implementation of Basic CPU Scheduling Algorithms – Non Preemptive [FCFS , SJF]

Expected Outcome of Experiment:

CO 2 Illustrate and analyse the Process, threads, process scheduling and thread scheduling

---

Books/ Journals/ Websites referred:

1. **Silberschatz A., Galvin P., Gagne G. "Operating Systems Principles", Willey Eight edition.**
2. **Achyut S. Godbole , Atul Kahate "Operating Systems" McGraw Hill Third Edition.**
3. **William Stallings, "Operating System Internal & Design Principles", Pearson.**
4. **Andrew S. Tanenbaum, "Modern Operating System", Prentice Hall.**

Pre Lab/ Prior Concepts:

Most systems handle numerous processes with short CPU bursts interspersed with I/O requests and a few processes with long CPU bursts. To ensure good time-sharing performance, a running process may be preempted to allow another to run. The ready list, or run queue, maintains all processes ready to run and not blocked by I/O or other system requests. Entries in this list point to the process control block, which stores all process information and state.

When an I/O request completes, the process moves from the waiting state to the ready state and is placed on the run queue. The process scheduler, a key component of the operating system, decides whether the current process should continue running or if another should take over. This decision is triggered by four events:

1. The current process issues an I/O request or system request, moving it from running to waiting.
2. The current process terminates.
3. A timer interrupt indicates the process has run for its allotted time, moving it from running to ready.
4. An I/O operation completes, moving the process from waiting to ready, potentially preempting the current process.

The scheduling algorithm, or policy, determines the sequence and duration of process execution, a complex task given the limited information about ready processes.

Description of the application to be implemented:

First-Come, First-Served Scheduling:

FCFS (First-Come, First-Served) is the simplest CPU scheduling algorithm where processes are executed in the order they arrive in the ready queue. It follows a FIFO approach, is non-preemptive, easy to implement, and ensures fairness, but may lead to higher waiting and turnaround times.

Code:

```
#include <iostream>

#include <vector>

using namespace std;



int main() {

    int n;
```

```
    cout << "Enter number of processes: ";
    cin >> n;

    vector<int>bt(n) , wt(n), tat(n);
    float avg_wt = 0, avg_tat = 0;

    cout << "Enter burst times:\n";
    for(int i = 0; i < n; i++) {
        cin >> bt[i];
    }

    wt[0] = 0;

    for(int i = 1; i < n; i++) {
        wt[i] = wt[i-1] + bt[i-1];
    }

    for(int i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
    }
```

```
    avg_wt /= n;
    avg_tat /= n;

    cout << "\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n";
    for(int i = 0; i < n; i++) {
        cout << "P" << i+1 << "\t" << bt[i] << "\t\t" << wt[i] << "\t\t"
<< tat[i] << endl;
    }

    cout << "\nAverage Waiting Time: " << avg_wt;
    cout << "\nAverage Turnaround Time: " << avg_tat;

    return 0;
}
```

```
Enter number of processes: 5
Enter burst times:
2
5
7
8
12

Process Burst Time      Waiting Time    Turnaround Time
P1      2               0               2
P2      5               2               7
P3      7               7               14
P4      8               14              22
P5      12              22              34

Average Waiting Time: 9
Average Turnaround Time: 15.8
```

Shortest job first :

Shortest Job First (SJF) is a CPU scheduling algorithm that selects the process with the smallest burst time for execution next. It can be preemptive or non preemptive and is known for minimizing average waiting time. However it may cause starvation of longer processes and requires estimation of burst time.

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;
    int at[n], bt[n], wt[n], tat[n], ct[n], done[n];
    cout << "Enter Arrival Time:\n";
    for(int i = 0; i < n; i++) {
        cin >> at[i];
        done[i] = 0;
    }
```

```
    cout << "Enter Burst Time:\n";
    for(int i = 0; i < n; i++) {
        cin >> bt[i];
    }
    int completed = 0, currentTime = 0;
    while(completed < n) {
        int idx = -1;
        int minBT = 1e9;
        for(int i = 0; i < n; i++) {
            if(at[i] <= currentTime && done[i] == 0) {
                if(bt[i] < minBT) {
                    minBT = bt[i];
                    idx = i;
                }
            }
        }
        if(idx != -1) {
            ct[idx] = currentTime + bt[idx];
            wt[idx] = ct[idx] - at[idx] - bt[idx];
            tat[idx] = ct[idx] - at[idx];

            currentTime = ct[idx];
            done[idx] = 1;
            completed++;
        } else {
            currentTime++;
        }
    }

    float avg_wt = 0, avg_tat = 0;

    cout << "\nP\tAT\tBT\tCT\tWT\tTAT\n";
    for(int i = 0; i < n; i++) {
        cout << "P" << i+1 << "\t" << at[i] << "\t" << bt[i] << "\t"
             << ct[i] << "\t" << wt[i] << "\t" << tat[i] << endl;
```

```
        avg_wt += wt[i];
        avg_tat += tat[i];
    }
    cout << "\nAverage Waiting Time: " << avg_wt/n;
    cout << "\nAverage Turnaround Time: " << avg_tat/n;
    return 0;
}
```

Output:

```
Enter number of processes: 5
Enter Arrival Time:
2
4
6
8
10
Enter Burst Time:
10
5
6
2
4

P       AT      BT      CT      WT      TAT
P1      2       10      12      0       10
P2      4       5       23      14      19
P3      6       6       29      17      23
P4      8       2       14      4       6
P5      10      4       18      4       8

Average Waiting Time: 7.8
Average Turnaround Time: 13.2
Process returned 0 (0x0)   execution time : 24.382 s
Press any key to continue.
```

Conclusion :

In conclusion, Shortest Job First scheduling improves system performance by minimizing average waiting and turnaround times through execution of shorter processes first. However, it may lead to starvation of longer processes and requires accurate estimation of burst times, making it efficient but less fair compared to simpler algorithms like FCFS.

Multiple-Choice Questions (MCQs)

1. **In FCFS scheduling, the process that arrives first:**

   A) Gets executed first

   B) Gets executed last

   C) Has the highest priority

   D) Has the shortest burst time

Ans: A) Gets executed first

2. **Which scheduling algorithm can cause the "Convoy Effect"?**

   A) FCFS

   B) SJF

   C) Round Robin

   D) Priority Scheduling

Ans: A) FCFS

3. **SJF scheduling algorithm is also known as:**

   A) Shortest Remaining Time First

   B) Shortest Time Next

   C) Longest Job First

A) Shortest Remaining Time First

Post Lab Descriptive Questions

1. Compare FCFS and SJF.
   FCFS executes processes in arrival order and is simple but often leads to higher waiting and turnaround times without starvation. SJF selects the shortest job

first, reducing waiting and turnaround times but may cause starvation of longer processes. SJF is more efficient but requires knowledge or estimation of burst times.

2. Discuss the impact of SJF scheduling on the average waiting time of processes.
Shortest Job First scheduling significantly reduces the average waiting time of processes by executing shorter jobs before longer ones, ensuring quick completion of small tasks. This minimizes queue delays and improves overall efficiency. However, longer processes may experience starvation if shorter jobs continuously arrive, making fairness a potential concern.

3. Consider a set of 4 processes with their respective arrival and burst times given in milliseconds.

Process	Arrival Time	Burst Time
P0	0	20
P1	15	25
P2	30	10
P3	45	15

A. **Apply the First-Come, First-Served (FCFS) and Shortest Job First (SJF) scheduling algorithms.**

B. **For each scheduling method:**
- Draw the Gantt Chart.
- Calculate the Average Turnaround Time.
- Calculate the Average Waiting Time.

C. Analyze the results and comment on the performance of FCFS and SJF for the given process set.

Batch: B2	Roll No.: 16010124107
Experiment No. 04	
Grade: AA / AB / BB / BC / CC / CD /DD	
Signature of the Staff In-charge with date	

FCFS

Process No.	AT	BT	CT	TAT	WT
P0	0	20	20	20	0
P1	15	25	45	30	5
P2	30	10	55	25	15
P3	45	15	70	25	10

Gantt chart.

0	20	45	55	70
P0	P1	P2	P3	

Avg TAT: (20 + 30 + 25 + 25) / 4 = 25
Avg WT : (0 + 5 + 15 + 10) / 4 = 7.5

SJF

Process No	AT	BT	CT	TAT	WT
P0	0	20	20	20	0
P1	15	25	45	30	5
P2	30	10	55	25	15
P3	45	15	70	25	10

Gantt chart:

0	20	45	55	70
P0	P1	P2	P3	

Avg TAT = 25
Avg WT = 7.5

analysis:

In this scenario, both FCFS and SJF deliver the same performance since processes arrive at different times, leaving no room for selection among multiple jobs. Consequently, SJF cannot reduce waiting time and behaves like FCFS. SJF shows improvement only when several processes are available simultaneously, which is not the case here.

Batch: B2	Roll No.: 16010124107
Experiment No. 04	
Grade: AA / AB / BB / BC / CC / CD /DD	
Signature of the Staff In-charge with date	

Date: 19/03/2026 **Signature of faculty in-charge**